National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 4: Data Augmentation Report

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 11, 2026

1 Introduction

Data augmentation is an essential technique in deep learning designed to artificially expand the size and diversity of a training dataset. By applying random but realistic transformations to existing images, the model is exposed to a wider variety of data points, which prevents it from memorizing specific training samples i.e. overfitting. This tutorial focuses on implementing a standard augmentation pipeline to introduce variability in spatial orientation, scale, and positioning using a single source image of a tropical bird.

2 Methodology

While the tutorial provided baseline implementations using Keras's `ImageDataGenerator`, this report utilizes the `torchvision.transforms` library within the PyTorch framework for consistency with advanced computer vision workflows.

2.1 Parameter Mapping

The augmentation pipeline was designed to replicate specific geometric transformations. Table 1 illustrates how the Keras parameters defined in the assignment were mapped to their PyTorch functional equivalents.

Table 1: Mapping Keras Augmentation Parameters to PyTorch Transforms

Transformation	**Keras Parameter**	**PyTorch Equivalent**
Rotation	`rotation_range=40`	`RandomRotation(degrees=40)`
Translation	`width/height_shift=0.2`	`RandomAffine(translate=(0.2, 0.2))`
Shearing	`shear_range=0.2`	`RandomAffine(shear=0.2)`
Zooming	`zoom_range=0.2`	`RandomAffine(scale=(0.8, 1.2))`
Flipping	`horizontal_flip=True`	`RandomHorizontalFlip(p=0.5)`

2.2 Implementation Snippet

The transformations were encapsulated in a `transforms.Compose` pipeline. The `RandomAffine` class was specifically leveraged to combine translation, shearing, and scaling into a single affine transformation step for computational efficiency.

Listing 1: PyTorch implementation of the Data Augmentation pipeline.

```
augmentation_pipeline = transforms.Compose([
transforms.RandomRotation(degrees=40),
transforms.RandomAffine(
degrees=0, translate=(0.2, 0.2), shear=0.2, scale
   =(0.8, 1.2)
),
transforms.RandomHorizontalFlip(p=0.5),
transforms.ToTensor(),
transforms.ToPILImage()
])
```

3 Results and Discussion

The original source image (Figure 1) was processed through the pipeline to generate a total of 40 unique augmented versions.

Figure 1: The original source image of the colorful tropical bird.

A representative sample of 20 images from the generated set is displayed in Figure 2. The resulting images demonstrate significant visual variability. The combination of **Rotation** and **Horizontal Flipping** ensures the model learns to recognize the bird from various orientations, while **Shifting** and **Zooming** simulate different camera distances and subject positioning within the frame.

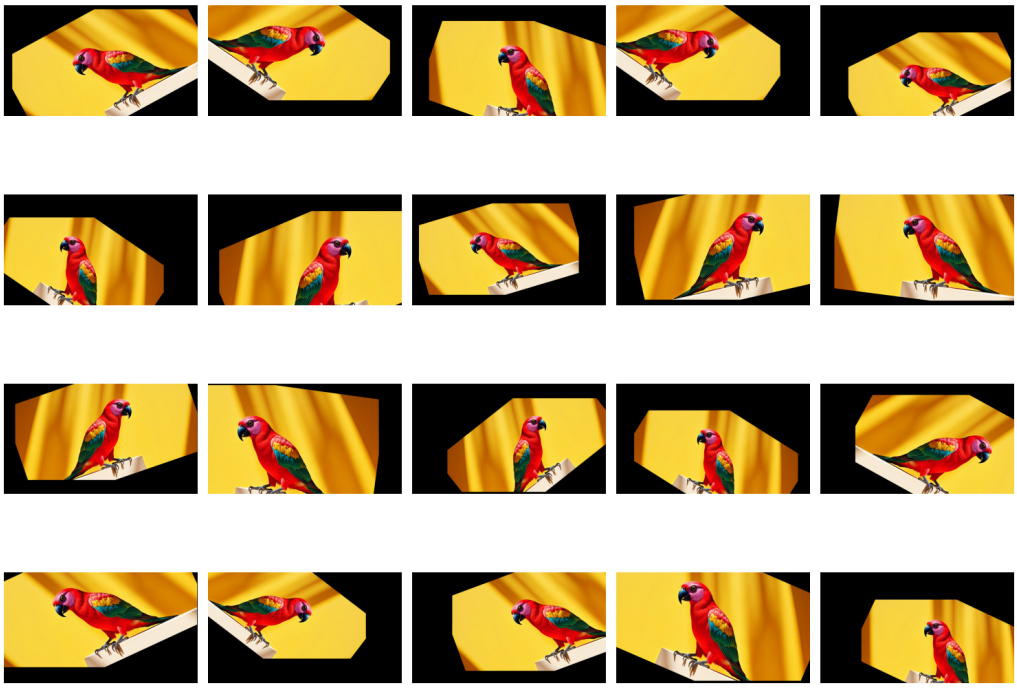

Figure 2: A 4x5 grid showing 20 randomly selected augmented samples, illustrating the diversity in scale, orientation, and shear.

4 Conclusion

The implementation of the data augmentation pipeline in PyTorch successfully fulfilled the assignment requirements. By mapping traditional Keras parameters to

`torchvision.transforms`, a robust dataset of 40 images was generated from a single source. This pipeline is highly efficient and scalable, providing a foundational preprocessing step for training Convolutional Neural Networks (CNNs) in future tasks.